

BotForge Arena

Project Brief

The idea

BotForge Arena is a multiplayer browser game in which players build robots, assign their robot a limited number of upgrade points, program its behaviour, and then let the robots fight automatically.

The important difference from a normal game is that the player does not control the robot during the battle.

The player controls the robot by designing its logic before the battle.

A good robot therefore does not necessarily win because it has the strongest weapon. It can win because it makes better decisions.

For example:

- Should it attack immediately?
- Should it keep distance from stronger enemies?
- Should it retreat when its health is low?
- Should it circle around obstacles?
- Should it fire only when its chance of hitting is high?
- Should it chase damaged enemies?
- Should it prioritize survival or damage?
- Should it remember where an enemy was last seen?

The final goal is to create an arena in which different programmable robots can compete automatically over many rounds.

Players should be able to upload a JSON file describing their robot.

The server validates the robot, loads it into the arena and runs the match.

The project should feel like a small version of a programming competition such as RoboCode, but presented as a polished modern browser game.

What you should learn

The purpose of this project is not just to create a game.

You should learn how software is divided into systems that work together.

During the project you will encounter concepts such as:

- variables
- numbers
- strings
- lists
- dictionaries
- JSON
- functions
- classes
- objects
- conditions
- if / else
- loops
- while
- state
- events
- game loops
- APIs
- WebSockets
- client/server architecture
- validation
- algorithms
- debugging
- Git
- working together on the same codebase

You are encouraged to use Claude Code extensively.

However, the goal is not to ask Claude Code:

“Build this entire game.”

Instead, divide the system into small problems, understand what each problem should do, let Claude Code help you implement it, and then inspect and test the result.

By the end of the project, you should be able to explain roughly how every major part of the system works.

PART A — BUILD THE GAME

1. Basic game concept

The game takes place in a top-down arena.

A match contains between two and eight robots.

Every robot has:

- a position
- a direction
- health
- armor
- movement speed
- weapon damage
- weapon accuracy
- weapon fire rate
- sensor range
- energy

Robots can:

- move forward
- move backward
- rotate
- scan for enemies
- select a target
- shoot
- retreat
- wait
- use defensive behaviour

The server simulates the battle.

The browser visualizes what is happening.

Players do not directly control their robots once the match begins.

2. Robot construction

Every robot receives exactly 100 build points.

The player must distribute these points between different attributes.

For example:

Speed

Determines how quickly the robot can move around the arena.

Armor

Determines how much damage the robot can survive.

Weapon Power

Determines how much damage each successful shot causes.

Accuracy

Determines how likely a shot is to hit the target.

Fire Rate

Determines how quickly the robot can shoot again.

Sensor Range

Determines how far the robot can detect enemies.

The total may never exceed 100 points.

It should therefore be impossible to build a robot that is best at everything.

A fast robot might have weak armor.

A heavily armored robot might move slowly.

A sniper might have excellent accuracy and weapon power but a low fire rate.

A scout might have excellent sensors and speed but weak weapons.

This creates the first interesting programming problem: different hardware requires different software strategies.

3. Start simple

The first version should not contain all features.

Initially, create only two robots with:

- position
- rotation
- health
- movement
- one weapon
- bullets
- collisions
- damage

At first, both robots can use extremely simple automatic behaviour.

For example:

Find nearest enemy.

Turn toward enemy.

Move toward enemy.

Shoot when enemy is close enough.

This version is deliberately simple.

If this works reliably, everything else can be built on top of it.

4. Recommended architecture

The game should run on one computer that acts as the server.

Other computers connected to the same Wi-Fi or LAN can open the server in their browsers.

The recommended architecture is:

Python backend:

FastAPI

The Python application owns the actual game state.

It decides where robots are, whether bullets hit, how much damage is caused and who wins.

WebSockets:

The browser maintains a live connection to the Python server.

The server continuously sends game-state updates to all connected browsers.

Browser frontend:

Use JavaScript together with Phaser.

Phaser is a browser game framework that makes sprites, animations, collisions, particles, cameras and user interfaces much easier than writing everything from scratch.

Database:

Use SQLite initially.

Store:

- users
- robots
- uploaded robot versions
- matches
- round results
- wins
- losses
- leaderboard scores

The important architectural rule is:

The Python server is authoritative.

A browser displays the battle, but it never decides who was hit or who won.

5. The game loop

Games are simulations that run repeatedly.

For example, the server might update the world 20 or 30 times every second.

Each update is called a tick.

During every tick the server roughly does this:

Read the current game state.

Give every robot the information its sensors are allowed to see.

Run the robot's decision logic.

Receive the robot's requested action.

Move the robots.

Move projectiles.

Check collisions.

Calculate damage.

Check whether robots have been destroyed.

Send the updated state to the browsers.

Repeat.

Understanding this loop is one of the most important learning goals of the project.

It is effectively one large loop running for the entire match.

6. Sensors

Bots should not automatically know everything happening in the arena.

They should receive information through sensors.

For example, a robot might know:

its own health

its own position

its own direction

its remaining energy

whether its weapon is ready

nearby walls

visible enemy robots

distance to visible enemies

direction to visible enemies

estimated enemy health

A robot with a sensor range of 250 pixels cannot see an enemy 400 pixels away.

This is important because it creates meaningful strategies.

A robot cannot simply ask the server where every enemy is.

7. Maps

After the basic arena works, introduce several maps.

Do not make increasingly difficult “levels”.

Instead create different environments requiring different strategies.

Examples:

Open Arena

Almost no cover. Speed and long-range weapons become valuable.

Pillars

Large objects create cover and block shots.

Crossfire

Several corridors connect through a dangerous central area.

Maze

Robots frequently lose sight of each other.

Island Arena

Several large obstacles create separate fighting zones.

Random Arena

Obstacle locations change before every match.

A good robot should ideally perform reasonably well on multiple maps.

Otherwise players could create a robot that only exploits one specific map.

8. Matches and rounds

One battle should not determine the winner.

Luck will influence individual fights.

Instead, a match should contain multiple rounds.

For example:

10 rounds.

Each round lasts a maximum of 60 seconds.

Spawn locations are randomized.

Map selection can change between rounds.

At the end of all rounds, the match result is calculated.

Possible scoring:

Winning a round: 3 points.

Surviving a draw: 1 point.

Destroying an enemy: additional combat statistics, but not necessarily leaderboard points.

This prevents one lucky shot from deciding everything.

9. Tournament mode

Once individual matches work, build a tournament system.

Players upload robots.

The server automatically creates matches between them.

For example:

Robot A vs Robot B

Robot A vs Robot C

Robot A vs Robot D

Robot B vs Robot C

Robot B vs Robot D

Robot C vs Robot D

Every pairing runs several rounds.

The results are then added to a leaderboard.

The tournament should be able to run automatically without anyone controlling the robots.

10. Leaderboard

The leaderboard should show more than wins and losses.

Possible statistics:

Rank

Robot name

Creator

Matches

Rounds won

Rounds lost

Win percentage

Damage caused

Damage received

Accuracy

Average survival time

Average distance travelled

Kills

A simple rating system can later be added so defeating a strong robot is worth more than defeating a weak robot.

11. The visual experience

The game should look much better than a typical beginner programming project.

The first visual version can still use simple shapes.

However, the final version should add:

animated robots

different robot chassis

rotating weapon turrets

projectile trails

muzzle flashes

explosions

sparks

damage effects

health bars

shields

shadows

arena lighting

movement trails

smooth camera movement

screen shake during large impacts

countdowns

round transitions

winner animations

A spectator should be able to understand what is happening without seeing the code.

The game should feel like watching tiny autonomous machines trying to outsmart each other.

12. Match analytics

After a fight, show a match report.

Examples:

Robot Alpha

Shots fired: 82

Hits: 37

Accuracy: 45%

Damage caused: 412

Damage received: 307

Distance travelled: 1,820

Enemies detected: 29

Average distance to target: 231

Time below 25% health: 8.3 seconds

This makes programming changes measurable.

Instead of saying:

"I think my new strategy is better."

A player can say:

"My accuracy increased from 31% to 47%, but survival time decreased."

This is much closer to real software development.

PART B — PROGRAM THE ROBOTS

13. The important idea

Players should not initially write Python code for their robot.

Allowing arbitrary Python to be uploaded would make the project much more complicated and potentially unsafe.

Instead, players program robots using JSON.

JSON contains structured information.

The game interprets that information and converts it into robot behaviour.

This creates a small programming language inside the game.

14. The robot file

A robot file contains three main parts.

Identity

Who is the robot?

Example information:

Name

Version

Creator

Build

How were the 100 hardware points distributed?

Logic

How should the robot behave?

A robot could conceptually contain information like this:

```
{ "name": "Hunter V3",  
  
  "build": { "speed": 20, "armor": 15, "weapon_power": 25, "accuracy": 20, "fire_rate": 10, "sensor_range":  
    10 },  
  
  "logic": [ ... ] }
```

The server must reject the robot if the build contains more than 100 points.

15. Conditions

The behaviour system should work using conditions.

For example:

IF enemy distance is less than 200

THEN shoot.

Or:

IF health is below 30%

THEN retreat.

Or:

IF no enemy is visible

THEN search.

The JSON system should eventually understand comparisons such as:

greater than

less than

equal to

not equal to

AND

OR

NOT

This teaches exactly the same logical concepts used by traditional programming languages.

16. Actions

The first available robot actions should be deliberately limited.

Examples:

move_forward

move_backward

turn_left

turn_right

turn_toward_enemy

move_toward_enemy

move_away_from_enemy

shoot

select_nearest_enemy

select_weakest_enemy

wait

scan

Later you can add more sophisticated actions.

Examples:

circle_target

move_to_cover

predict_target_position

remember_position

patrol

strafe

17. Priority rules

Robot logic should run from the highest-priority rule to the lowest-priority rule.

Imagine this strategy:

Priority 1

If health is below 20%, retreat.

Priority 2

If an enemy is extremely close, shoot.

Priority 3

If an enemy is visible, move toward it.

Priority 4

If no enemy is visible, search.

This introduces an important programming concept:

Order matters.

Consider a badly written robot:

If an enemy is visible, attack.

If health is below 10%, retreat.

The robot may never reach the retreat instruction because the attack instruction always happens first.

Changing the order can completely change the behaviour.

18. The implicit loop

The game itself repeatedly evaluates the robot logic.

That means a player does not initially need to manually create an infinite loop.

Every game tick effectively performs:

Observe.

Think.

Act.

Observe.

Think.

Act.

Repeat.

Later, introduce explicit loop concepts.

For example, a robot behaviour could mean:

While the enemy is closer than 150:

Move backward.

Keep weapon pointed at enemy.

Shoot whenever weapon is ready.

Players now begin to understand why careless loops can be dangerous.

19. Safe while loops

The robot interpreter must never allow a player's logic to freeze the server.

Consider:

WHILE true:

 turn_left

This could theoretically run forever.

The interpreter must therefore have a maximum number of instructions it will execute during each game tick.

For example:

Maximum 50 logic operations per robot per tick.

If the robot exceeds the limit, stop its program for that tick.

The match continues.

Record an error such as:

Logic execution limit reached.

This introduces an important real-world programming problem:

Infinite loops.

20. State and memory

The next major upgrade should allow robots to remember information.

For example:

last_enemy_position

last_enemy_seen_time

current_target

previous_health

shots_fired

shots_hit

Players could then create behaviour such as:

If an enemy disappears behind a wall, move toward the last known position.

If you recently received damage but cannot see an enemy, move away from the direction the damage came from.

If the current target has not been seen for five seconds, search for another target.

This introduces the programming concept of state.

The robot is no longer reacting only to the current moment.

It remembers what happened previously.

21. Variables

Eventually allow a small number of user-defined variables.

Examples:

aggression = 70

preferred_distance = 300

retreat_health = 25

A player can then modify these numbers instead of rewriting the entire strategy.

This makes experimentation easy.

Version 1:

preferred_distance = 150

Version 2:

preferred_distance = 250

Version 3:

preferred_distance = 350

Run each version through 100 simulated rounds.

Compare the results.

This introduces a simplified version of parameter optimization.

22. Example strategies

Players should quickly discover that very different strategies are possible.

The Tank

High armor.

High weapon damage.

Low movement speed.

Moves toward the closest enemy and keeps firing.

The Sniper

High sensor range.

High accuracy.

High weapon damage.

Low armor.

Tries to maintain a large distance.

The Mosquito

Very high speed.

High fire rate.

Low damage.

Continuously circles opponents.

The Coward

Very defensive.

Attacks damaged enemies.

Runs away from strong enemies.

Often survives until other robots eliminate each other.

The Hunter

Targets the weakest visible enemy.

Changes targets frequently.

Attempts to finish damaged robots.

The Ambusher

Uses obstacles.

Avoids open areas.

Attacks enemies when they come close.

There does not need to be one perfect strategy.

That is what makes the project interesting.

23. Information should have a cost

Advanced bots should not automatically receive perfect information.

For example, detecting an enemy might give:

distance

direction

estimated health

But perhaps not:

exact weapon strength

exact armor

the enemy's complete program

This creates uncertainty.

Later versions can introduce sensor upgrades that reveal more information.

24. Energy

Once the basic combat system works well, introduce energy.

Movement consumes energy.

Scanning consumes energy.

Shooting consumes energy.

Energy slowly regenerates.

This creates more complicated strategies.

A robot that constantly scans and shoots might run out of energy.

A smarter robot could stop firing when accuracy is too low.

This enables logic such as:

If energy is below 20%, stop attacking.

If the enemy is farther than 400, do not shoot.

If weapon accuracy is currently poor, move closer first.

This makes efficient decision-making valuable.

25. Programming progression

Do not introduce the complete robot language at once.

Stage 1 should only support simple rules.

IF condition THEN action.

Stage 2 introduces multiple conditions.

IF condition AND condition THEN action.

Stage 3 introduces priorities.

Stage 4 introduces ELSE.

Stage 5 introduces variables.

Stage 6 introduces memory.

Stage 7 introduces WHILE.

Stage 8 introduces sequences of actions.

Stage 9 introduces reusable behaviours.

For example:

Behaviour: retreat

Behaviour: attack

Behaviour: patrol

Behaviour: search

Eventually a bot could combine behaviours instead of containing hundreds of individual instructions.

This progression mirrors the way people learn programming.

26. Robot debugger

One of the most useful features should be a live robot debugger.

When watching a match, clicking a robot should show what it is currently thinking.

For example:

Current target:

Robot Gamma

Enemy distance:

283

Health:

42%

Energy:

71%

Current rule:

Rule 4 — Maintain preferred distance

Current action:

Move backward

Weapon:

Ready

This would create a major “wow” effect while also making debugging dramatically easier.

Instead of only seeing the robot move backward, the player sees why it decided to move backward.

27. Logic visualization

An even better version should visualize the currently active JSON rule.

Imagine a side panel displaying the robot's rules.

While the battle runs, the rule currently being executed lights up.

The player can watch:

Rule 1

Enemy close? No.

Rule 2

Health low? No.

Rule 3

Enemy visible? Yes.

Action:

Move toward enemy.

This makes abstract programming logic visually understandable.

28. Replay mode

Matches should eventually be recordable.

Instead of recording video, save game events.

For example:

Robot positions.

Shots.

Hits.

Damage.

Destroyed robots.

Important decisions.

The browser can later replay those events.

This allows players to inspect why their robot lost.

A timeline could allow them to jump to:

First contact.

First hit.

Robot below 50% health.

Robot below 20% health.

Final kill.

29. Simulation mode

The normal game should show battles visually.

However, there should eventually also be a fast simulation mode.

In simulation mode the game does not need to animate every frame.

It could run hundreds of matches much faster.

For example:

Hunter V4 vs Tank V7

Run 500 rounds.

Results:

Hunter wins: 312

Tank wins: 176

Draws: 12

This makes strategy testing much more interesting.

The player can now make a change to their logic and statistically evaluate whether it improved the robot.

30. Tournament night

The final project should support a classroom or team tournament.

Every participant submits one robot.

Submissions close.

Nobody can modify their bot anymore.

The server automatically creates the tournament.

The browser displays the battles.

A leaderboard updates live.

The final two robots compete in a longer championship match.

This gives the project a clear final event rather than simply ending when the software is finished.

DEVELOPMENT MILESTONES

Milestone 1 — Moving Robots

Create a browser arena.

Display two robots.

The Python server controls their positions.

The browser receives updates.

Robots can move and rotate.

No weapons yet.

Success means that two browsers on the same network can open the game and see the same robots moving.

Milestone 2 — Combat

Add:

projectiles

health

damage

robot destruction

round start

round end

random spawn positions

At this stage the robots can still use hard-coded behaviour.

Milestone 3 — Robot Builder

Create a page where a player can name a robot and distribute 100 build points.

Validate the result on the Python server.

Save the robot.

Display a visual robot card showing its statistics.

Milestone 4 — JSON Upload

Allow the player to upload a robot JSON file.

Validate:

correct JSON

required fields

valid actions

valid conditions

maximum 100 build points

valid robot name

logic complexity limits

Show understandable error messages when something is wrong.

Milestone 5 — Bot Interpreter

Create the system that reads the JSON logic.

Start with:

conditions

priority

actions

Then make the robots fight using uploaded logic instead of hard-coded Python behaviour.

This is the most important milestone in the entire project.

Milestone 6 — Multiplayer Lobby

Players on the same network should be able to:

open the website

choose a name

upload their robot

see other connected players

mark themselves ready

start a match

spectate together

Milestone 7 — Multiple Rounds

Run several rounds automatically.

Reset the arena between rounds.

Randomize spawn locations.

Record results.

Display the current score.

Milestone 8 — Leaderboard

Store permanent statistics.

Show rankings.

Allow robot versions such as:

Hunter v1

Hunter v2

Hunter v3

Players should be able to see whether later versions actually perform better.

Milestone 9 — Visual Polish

Now add the features that make the project feel impressive.

Particles.

Animations.

Sound.

Explosions.

Robot skins.

Weapon effects.

Arena backgrounds.

Match intro.

Winner screen.

Spectator camera.

Robot debugger.

Milestone 10 — Tournament System

Automatically schedule many robots against each other.

Run matches.

Calculate results.

Produce the final leaderboard.

HOW TO WORK WITH CLAUDE CODE

Claude Code should be treated like another developer on the team.

Do not give it the entire project and hope that everything works.

Work on one clearly defined problem at a time.

A good task might be:

Create a Python Robot class containing position, direction, health and movement speed. Add tests showing that taking damage reduces health and health can never become negative.

A bad task would be:

Build the robot game.

Before asking Claude Code to implement something, you should be able to explain in plain English what the feature should do.

After Claude Code changes the project, inspect what changed.

Ask it to explain unfamiliar parts.

Run the software.

Try to break the feature.

If it does not behave correctly, describe exactly what you expected and what actually happened.

Do not immediately ask it to rewrite everything.

Debug the specific problem.

TEAMWORK

The project can be divided between several people.

One person can primarily work on the Python simulation.

Another can work on the browser visualization.

Another can work on robot JSON validation and the interpreter.

Another can work on multiplayer, leaderboard and match management.

However, nobody should exclusively understand one part of the project.

Regularly explain your work to each other.

Use Git branches for separate features.

Create small commits.

Avoid having several people simultaneously changing the same large file.

Before implementing a large feature, agree on what information must pass between the systems.

For example, the frontend and backend must agree on what a robot update looks like.

This is exactly the type of coordination required in real software projects.

IMPORTANT DESIGN RULES

The game server always decides the truth.

Uploaded bots never execute arbitrary Python.

Robot logic is intentionally limited.

Every robot receives the same amount of build points.

Robot programs have execution limits.

Every match should be reproducible when possible.

Game rules should be separated from visual effects.

Robot logic should be separated from robot hardware statistics.

The browser should never need to understand the internal implementation of the bot program.

The first version should be simple enough to finish.

Advanced features should be added only when the previous milestone works reliably.

OPTIONAL ADVANCED FEATURES

Once the main game works, there are many possibilities.

Different weapons.

Mines.

Missiles.

Shields.

Radar.

Limited ammunition.

Teams.

Capture-the-flag mode.

King-of-the-hill mode.

Moving obstacles.

Destructible walls.

Fog of war.

Random map generation.

Bot alliances.

Communication between team robots.

Evolutionary optimization.

Automatic tournaments.

Heat maps.

Advanced match analytics.

3D rendering with Three.js.

An AI commentator describing the battle.

A “code view” explaining why robots are making decisions.

Do not begin with these features.

They are rewards for getting the core architecture right.

THE FINAL EXPERIENCE

The finished version should work approximately like this.

A player opens BotForge Arena.

They create a robot.

They have 100 hardware points.

They decide whether their robot should be fast, defensive, accurate, powerful or something in between.

They write a JSON program describing how it should behave.

They upload the robot.

The server checks the program.

Other players do the same.

Everyone joins the arena.

A countdown appears.

3.

2.

1.

DEPLOY.

The robots enter the arena.

Nobody controls them anymore.

Every robot follows only the logic its creator programmed.

Players watch their decisions live.

They can click their robot and see what rule is currently active.

The robots fight.

One survives.

A new round automatically begins.

After several rounds, match statistics appear.

After many matches, the leaderboard changes.

Players return to their JSON file.

They change one condition.

They adjust one variable.

They upload version 2.

And they immediately want to know:

Is my robot smarter now?

That question is the core of the entire project.